

# Assignment #3

## Paxos-based Key/Value Service

---

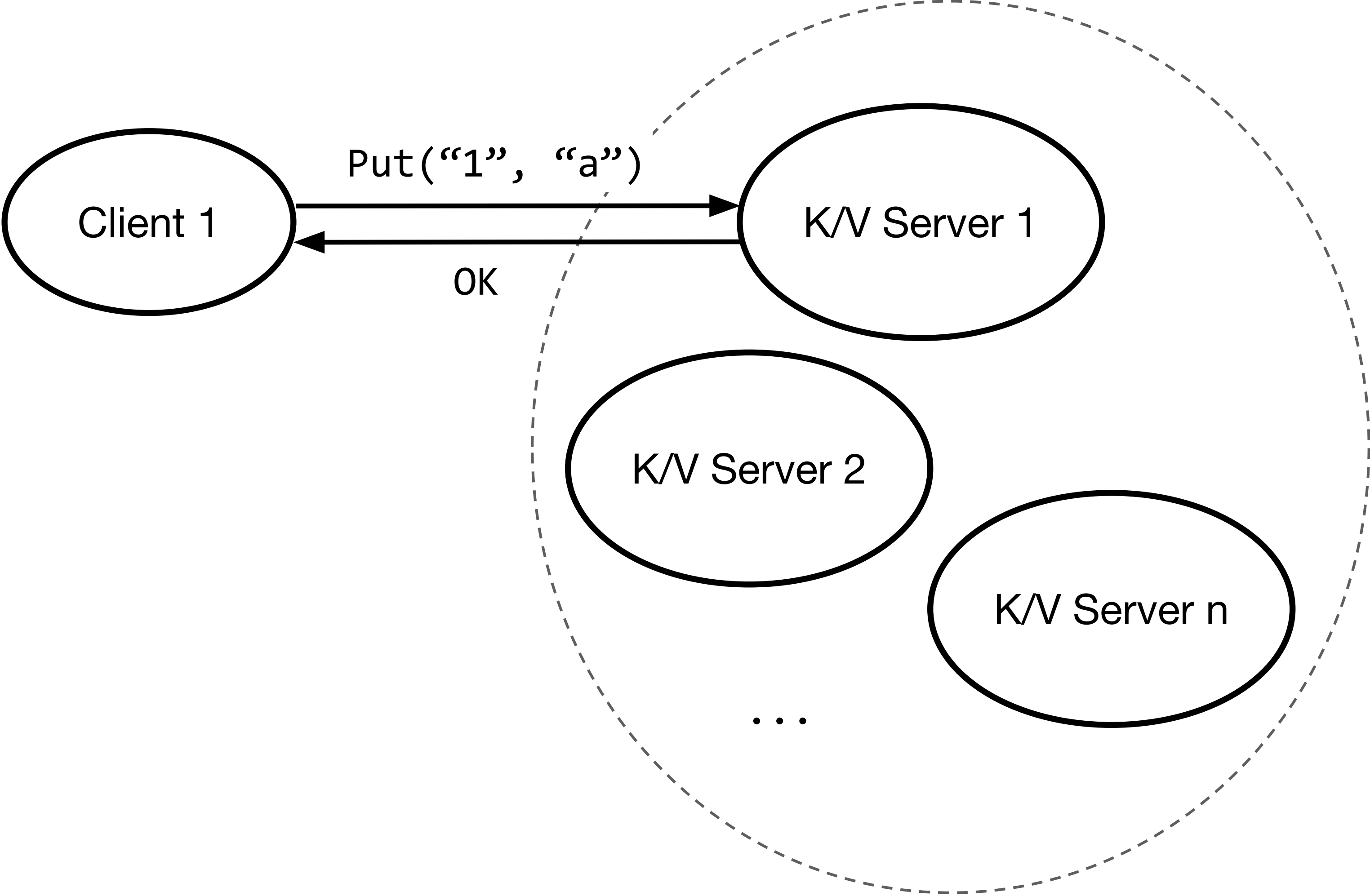
Xijiao Li (xl2950)

Spring 2022

# Motivation

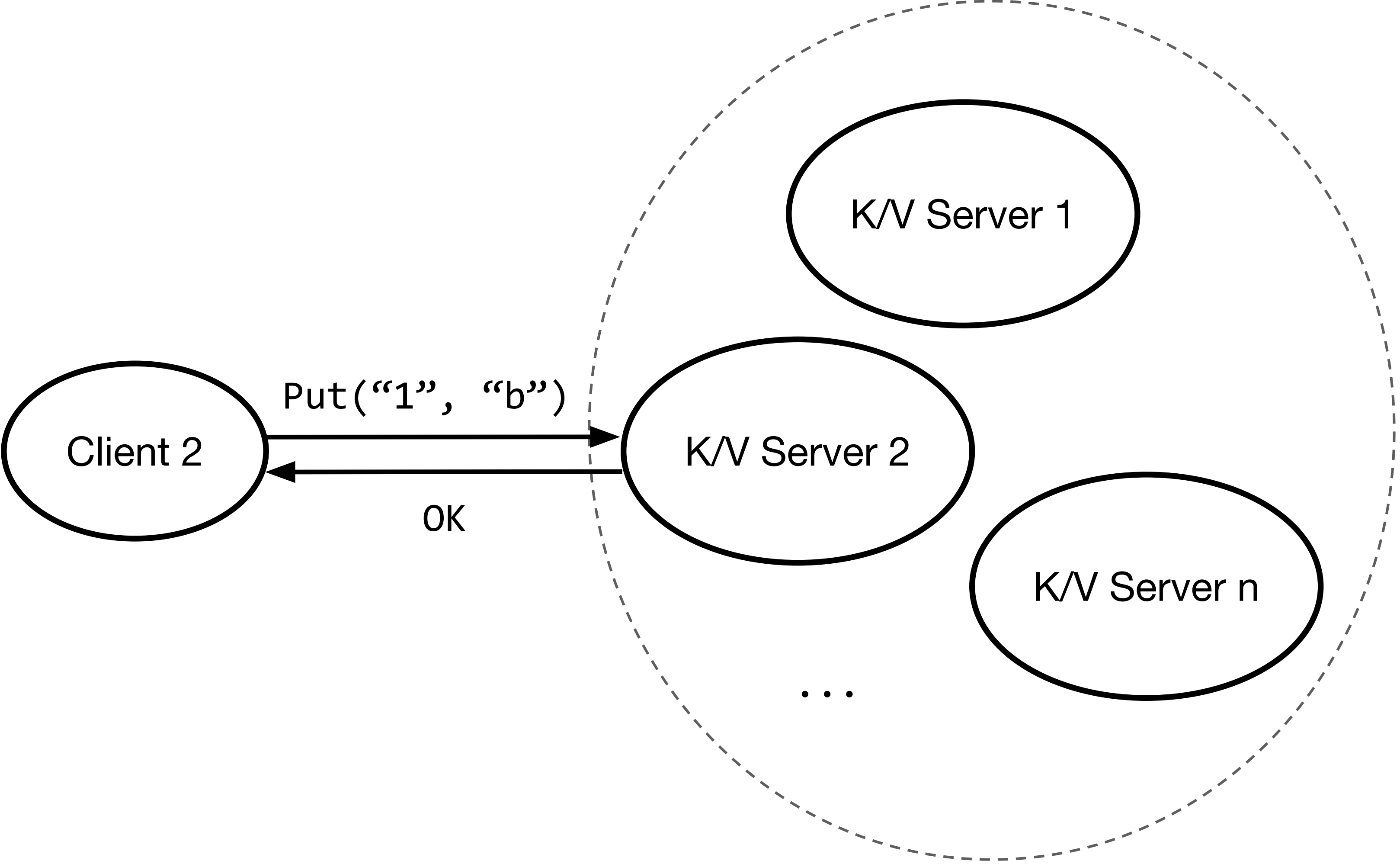
- Primary/backup Key/Value Service has a few limitations:
  - can only tolerate one replica failure
  - need to have a fault-tolerant view service
- Paxos-based Key/Value Service:
  - No master view server.
  - A set of servers (replicas) will process all client requests.
  - Can continue to process client operations as long as we can assemble a majority of replicas.

# Paxos-based Key/Value Service

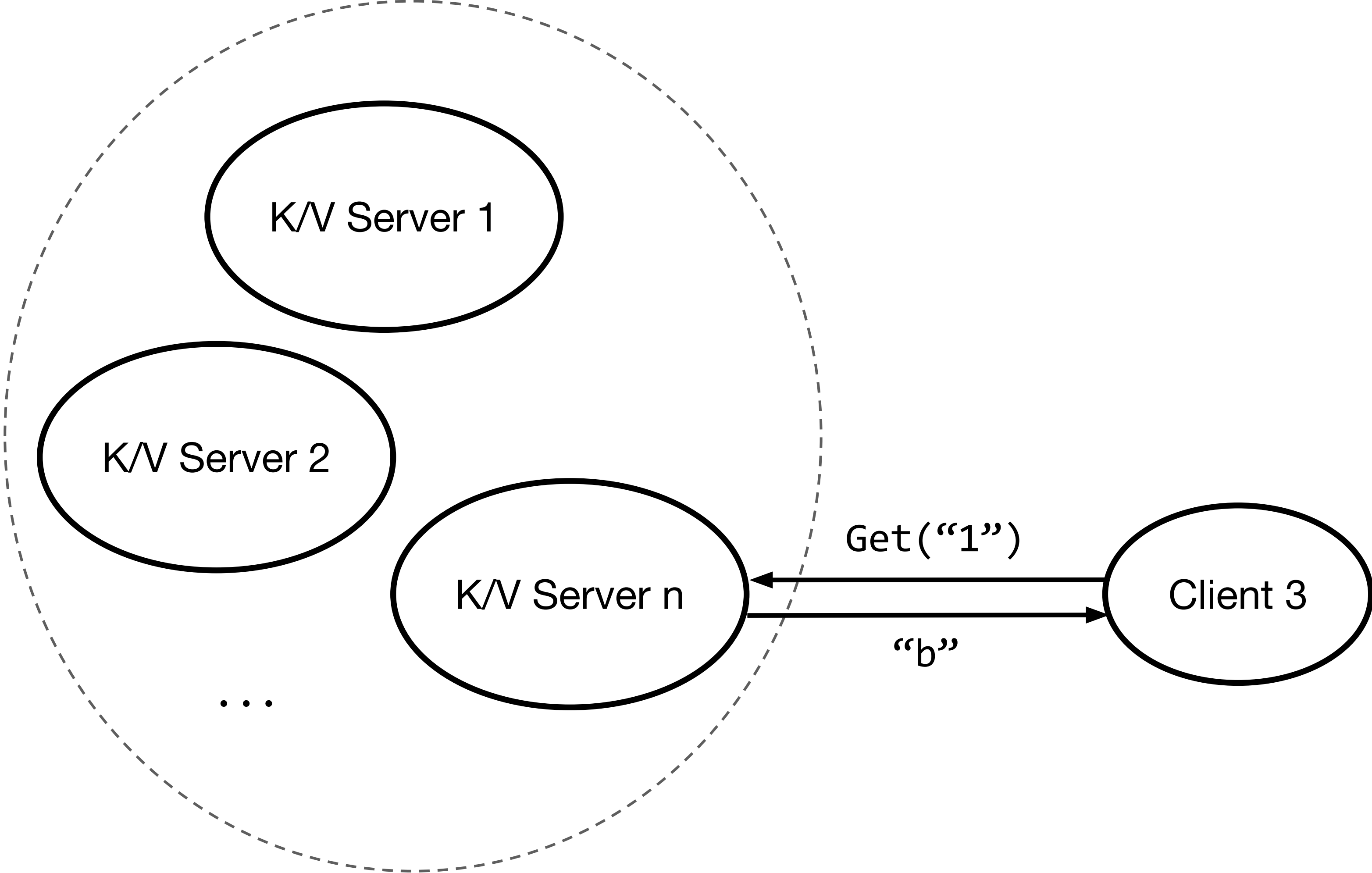




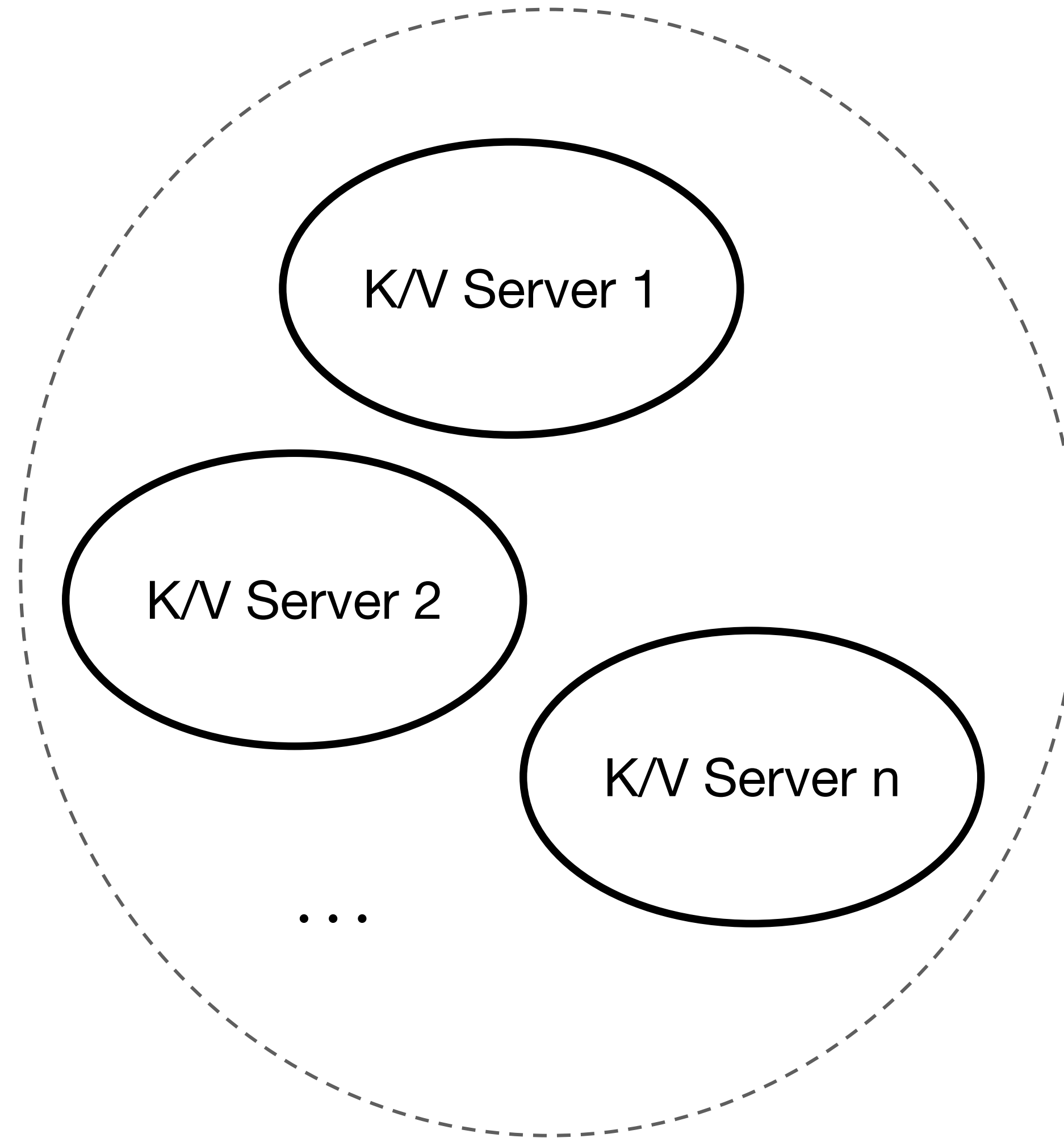
# Paxos-based Key/Value Service



# Paxos-based Key/Value Service



# Paxos-based Key/Value Service

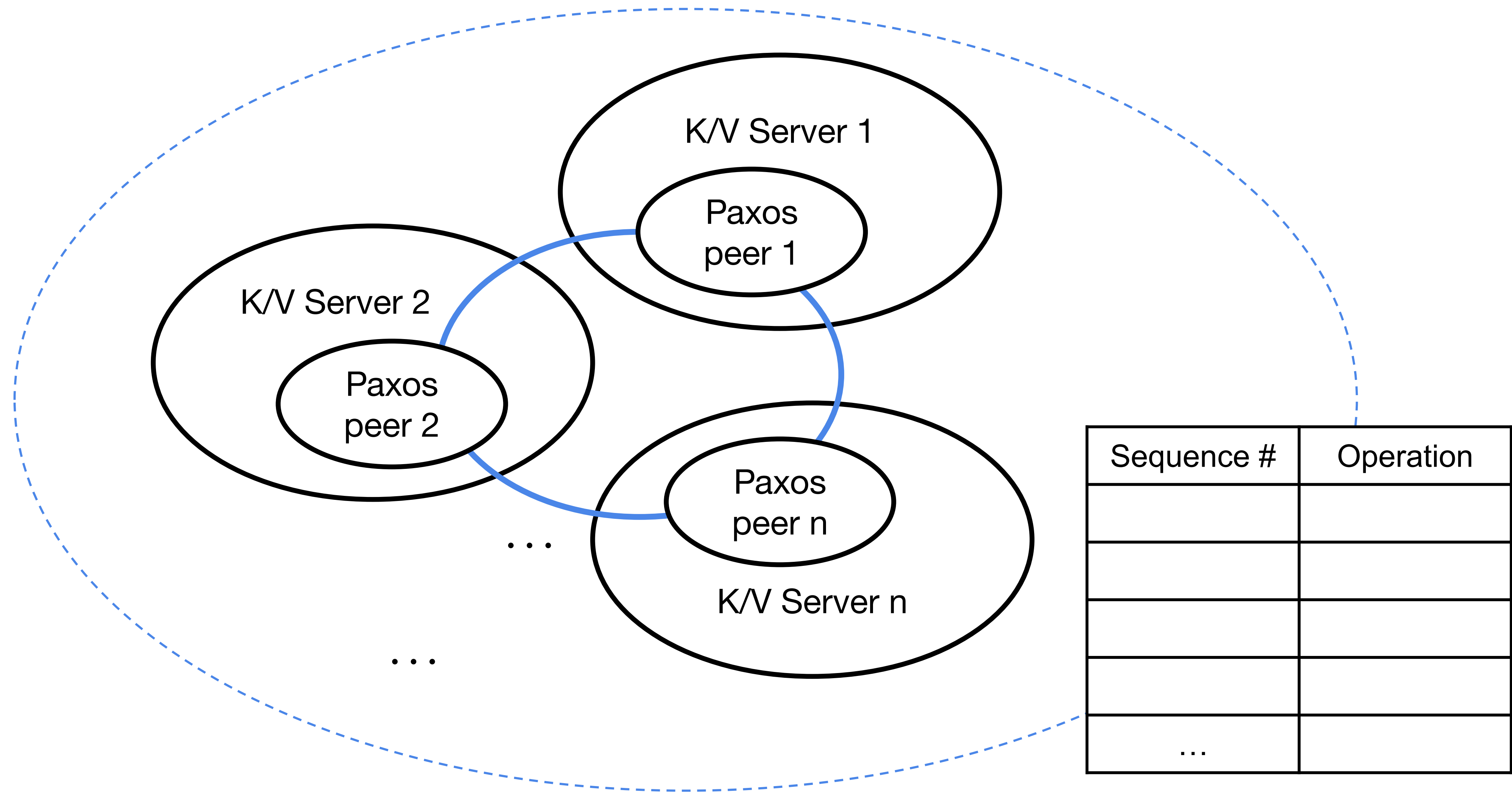




# Using consensus

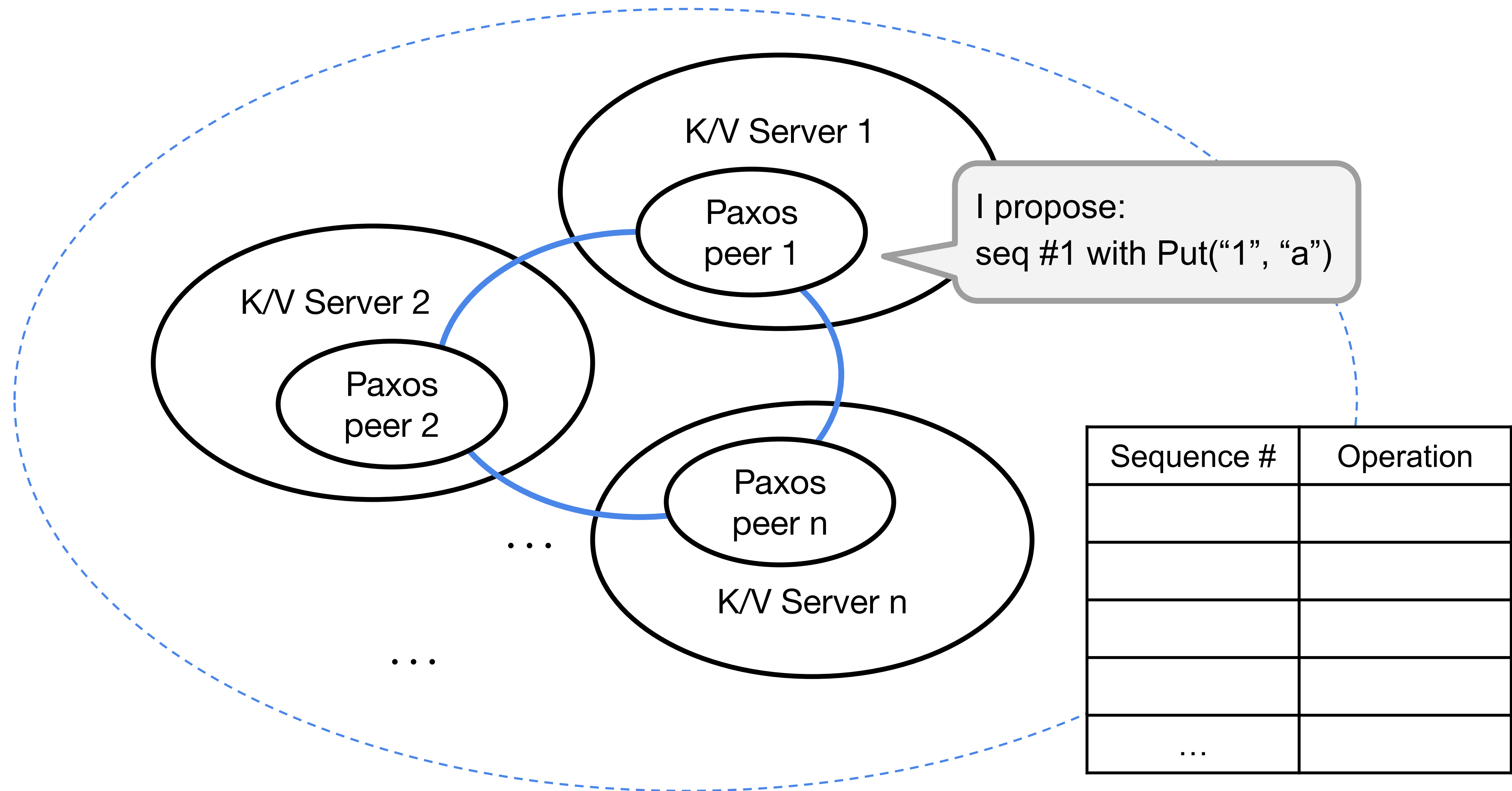
- Replicas maintain a *log* of operations.
  - Clients can send requests to any k/v servers.
  - Upon receiving a request, k/v server proposes it as next entry in log.
  - Run consensus among servers.
  - Once consensus completes, execute op in log and return to client.
- Paxos is a kind of protocols for *solving consensus* in a network of unreliable or fallible servers.
  - Put in plain English, it's trying to solve a problem that a system will only choose one value when one or more servers can propose different value(s).

# Paxos-based Key/Value Service

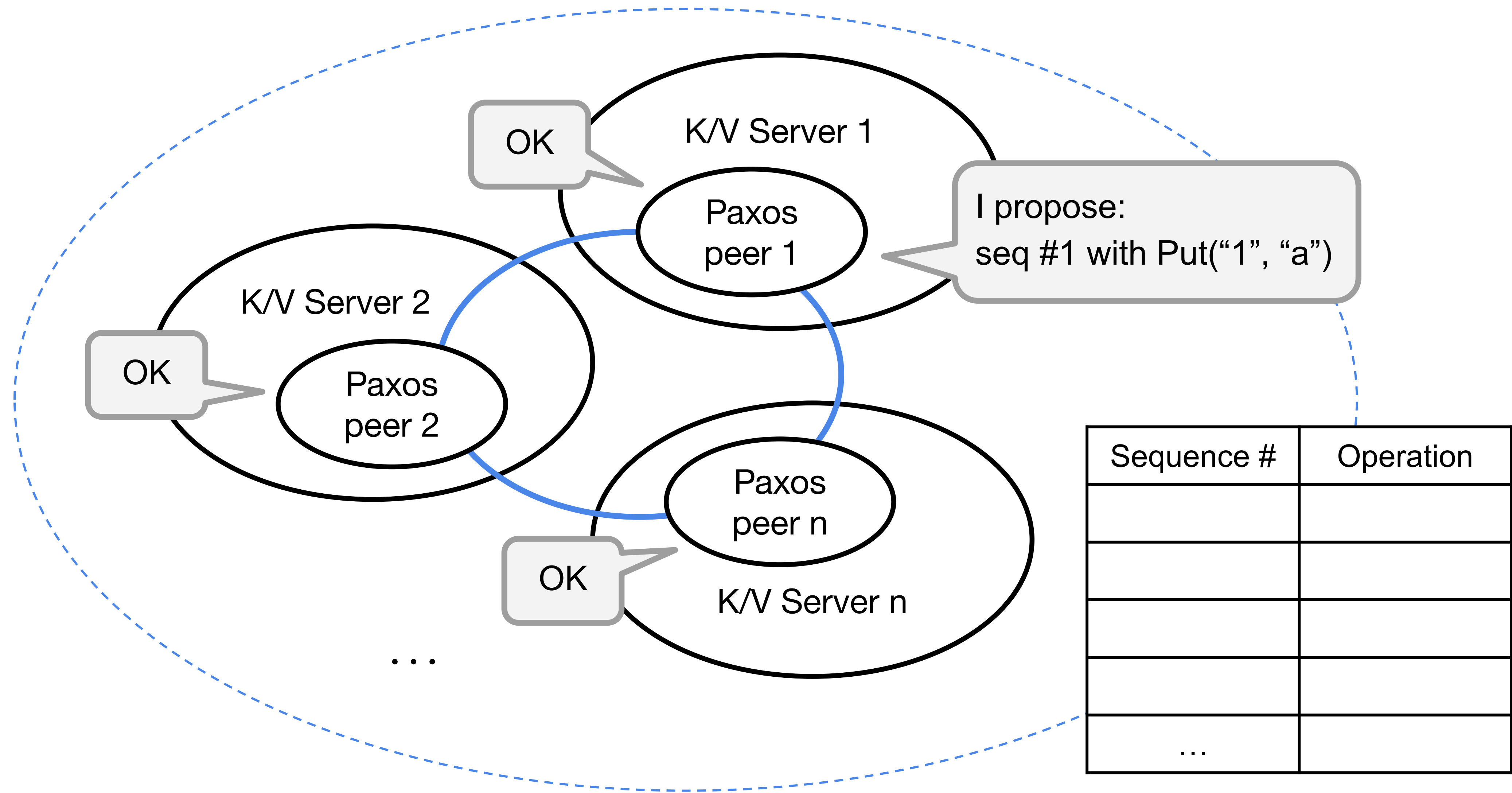




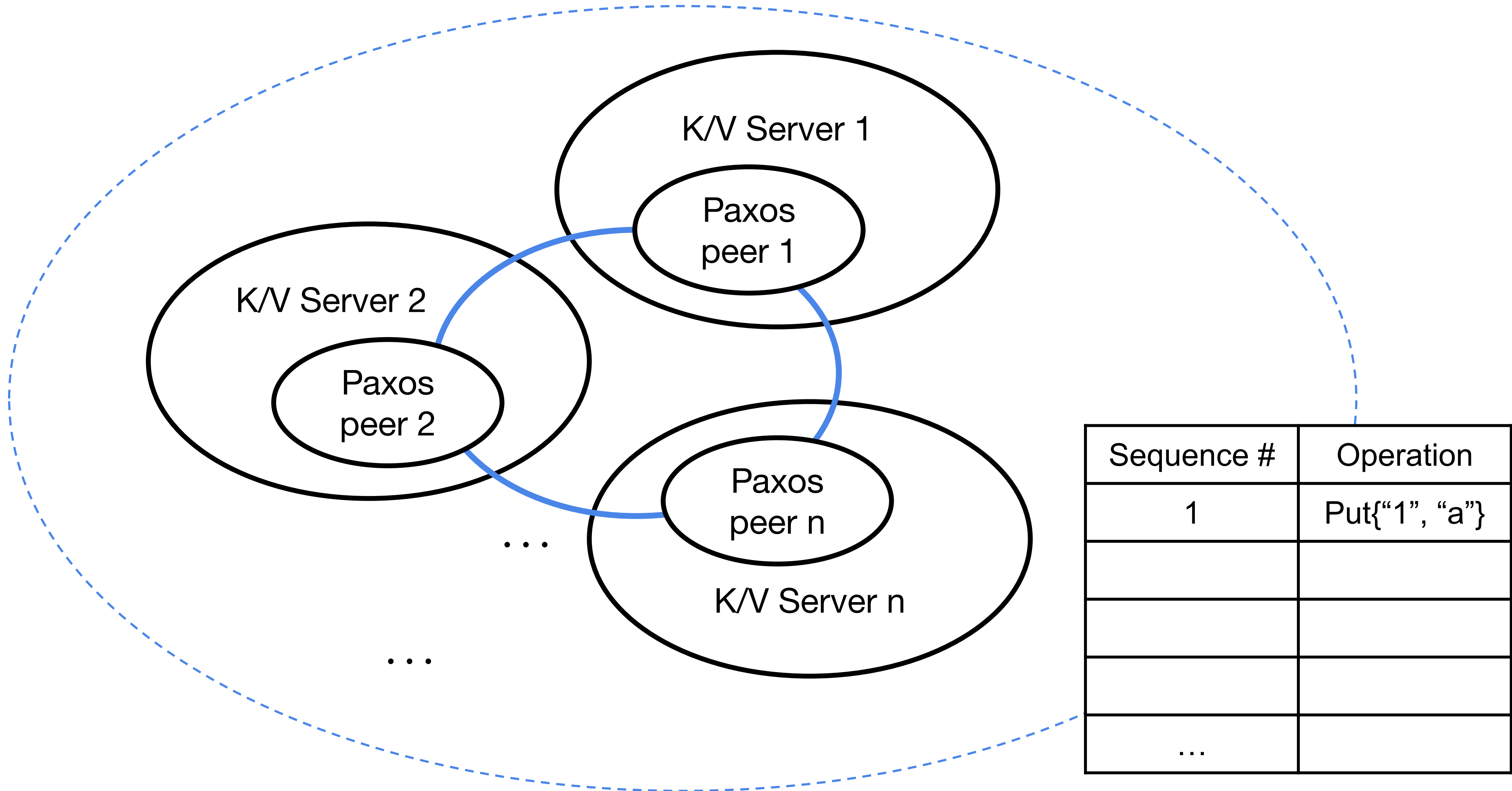
# Paxos-based Key/Value Service



# Paxos-based Key/Value Service

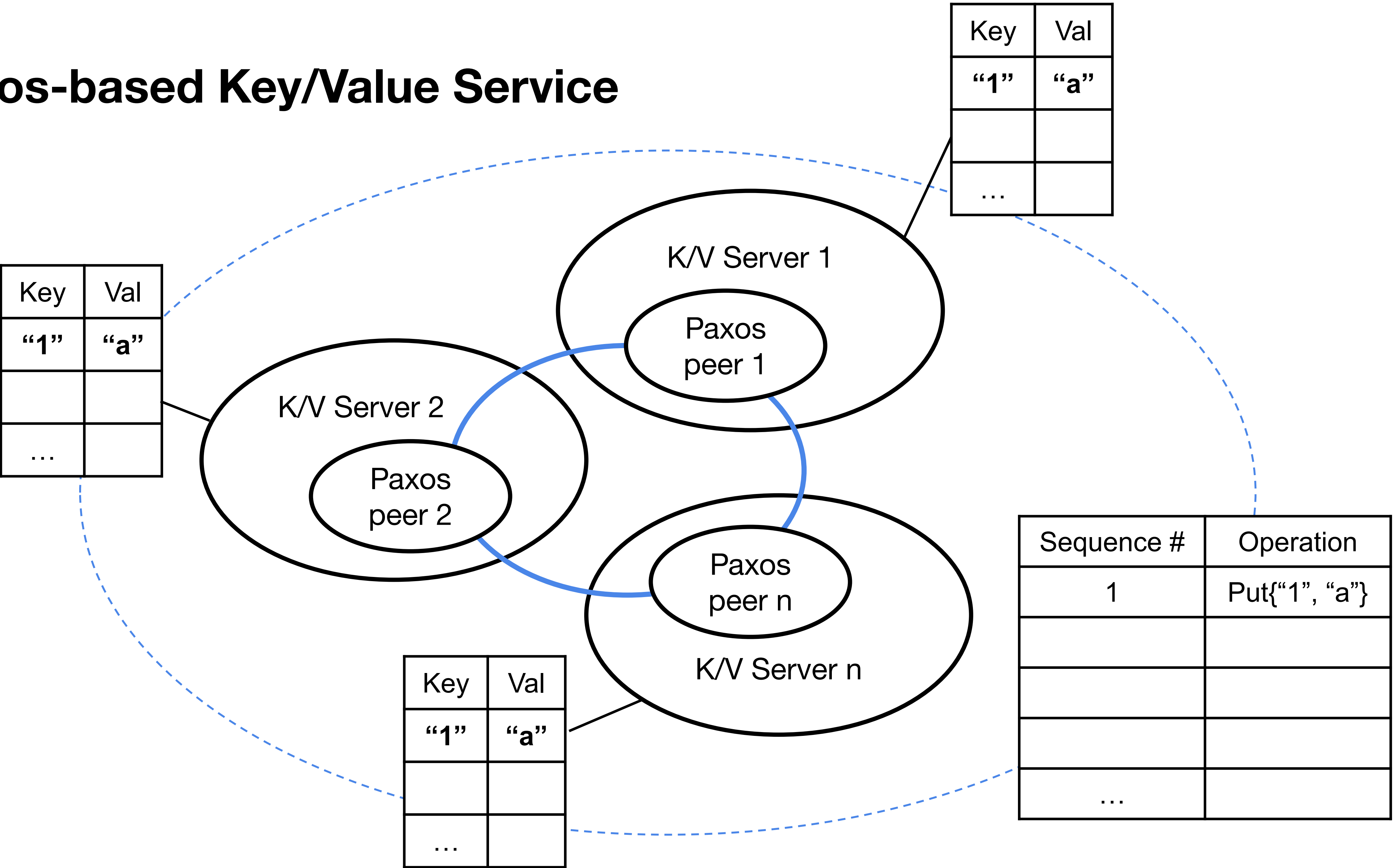


# Paxos-based Key/Value Service

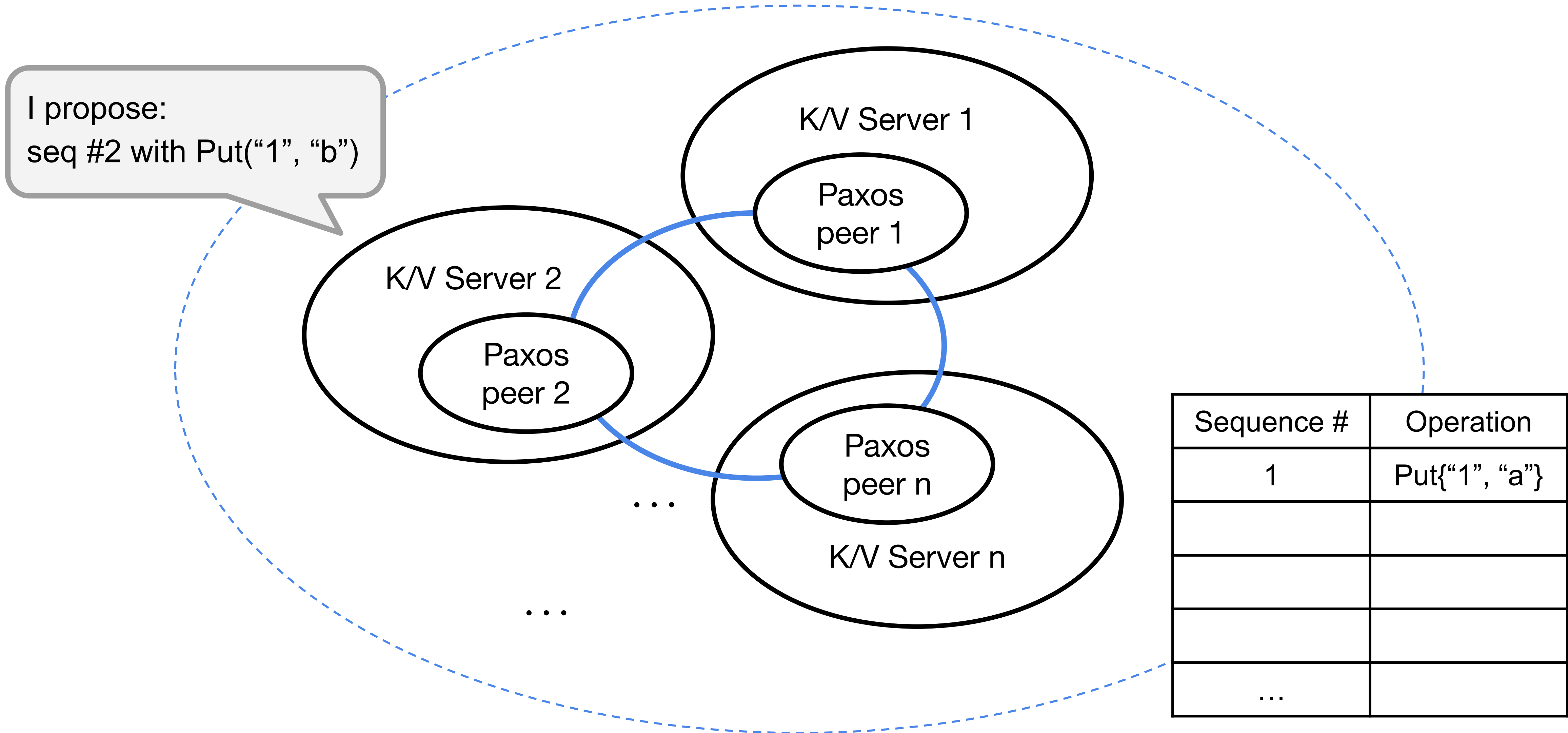




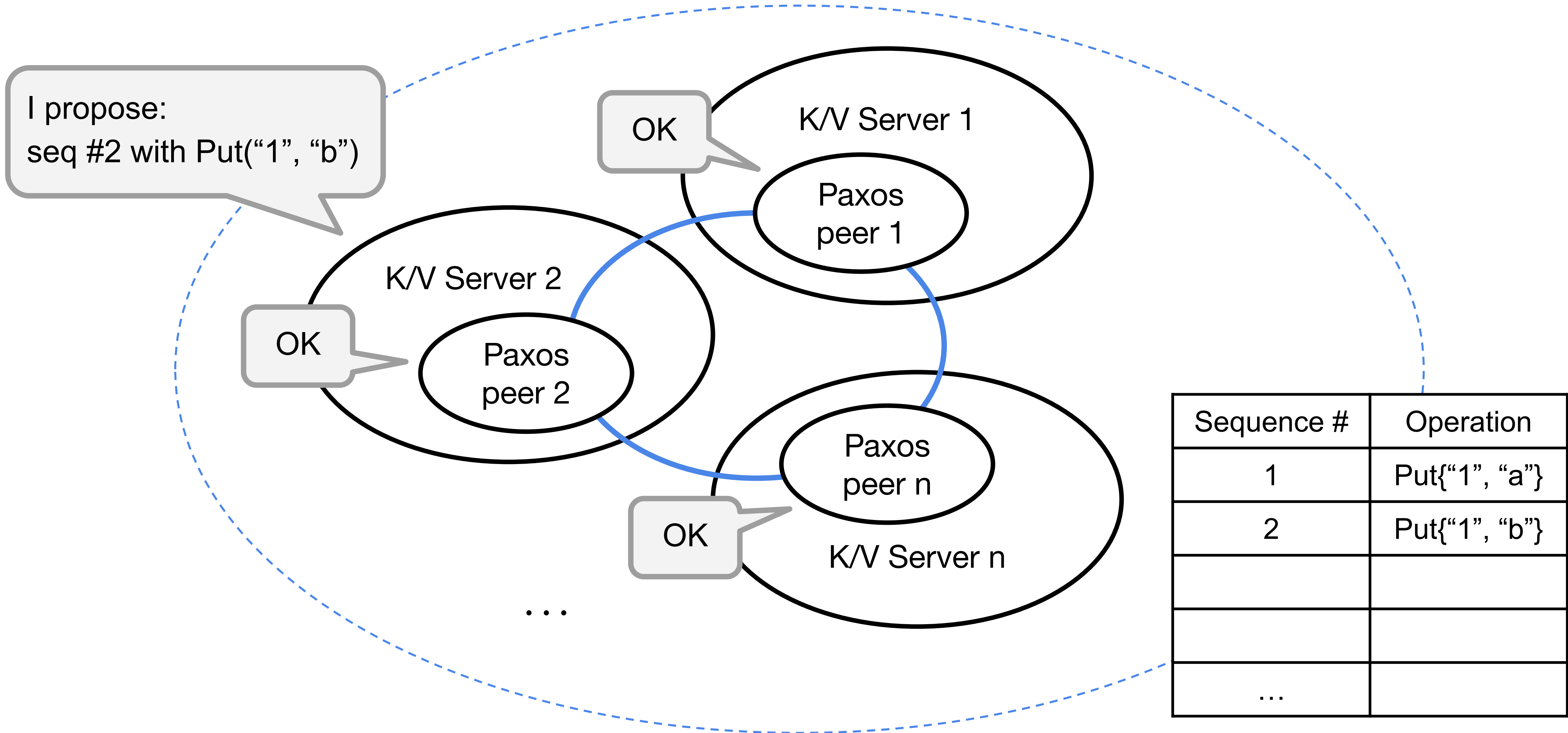
# Paxos-based Key/Value Service



# Paxos-based Key/Value Service

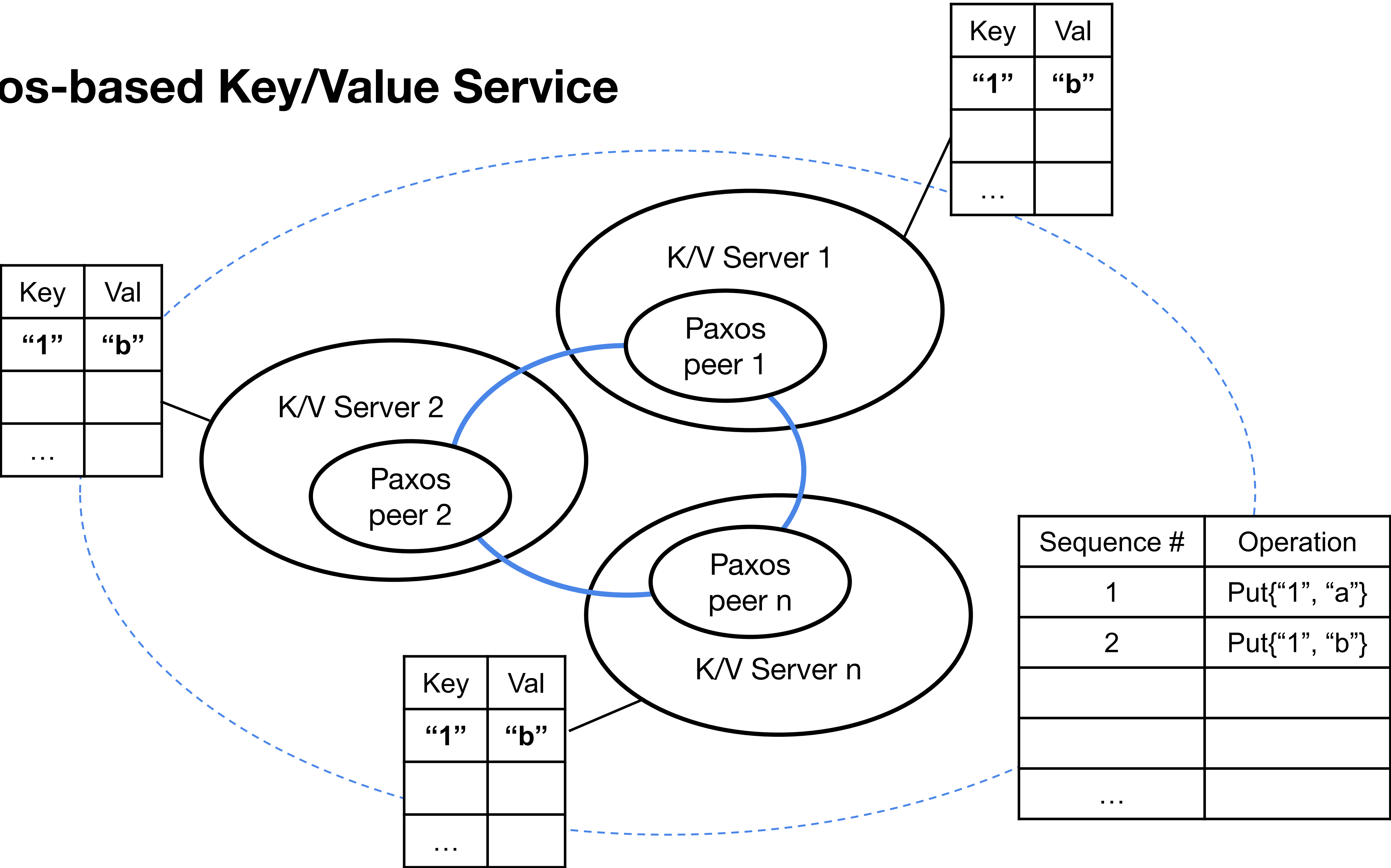


# Paxos-based Key/Value Service



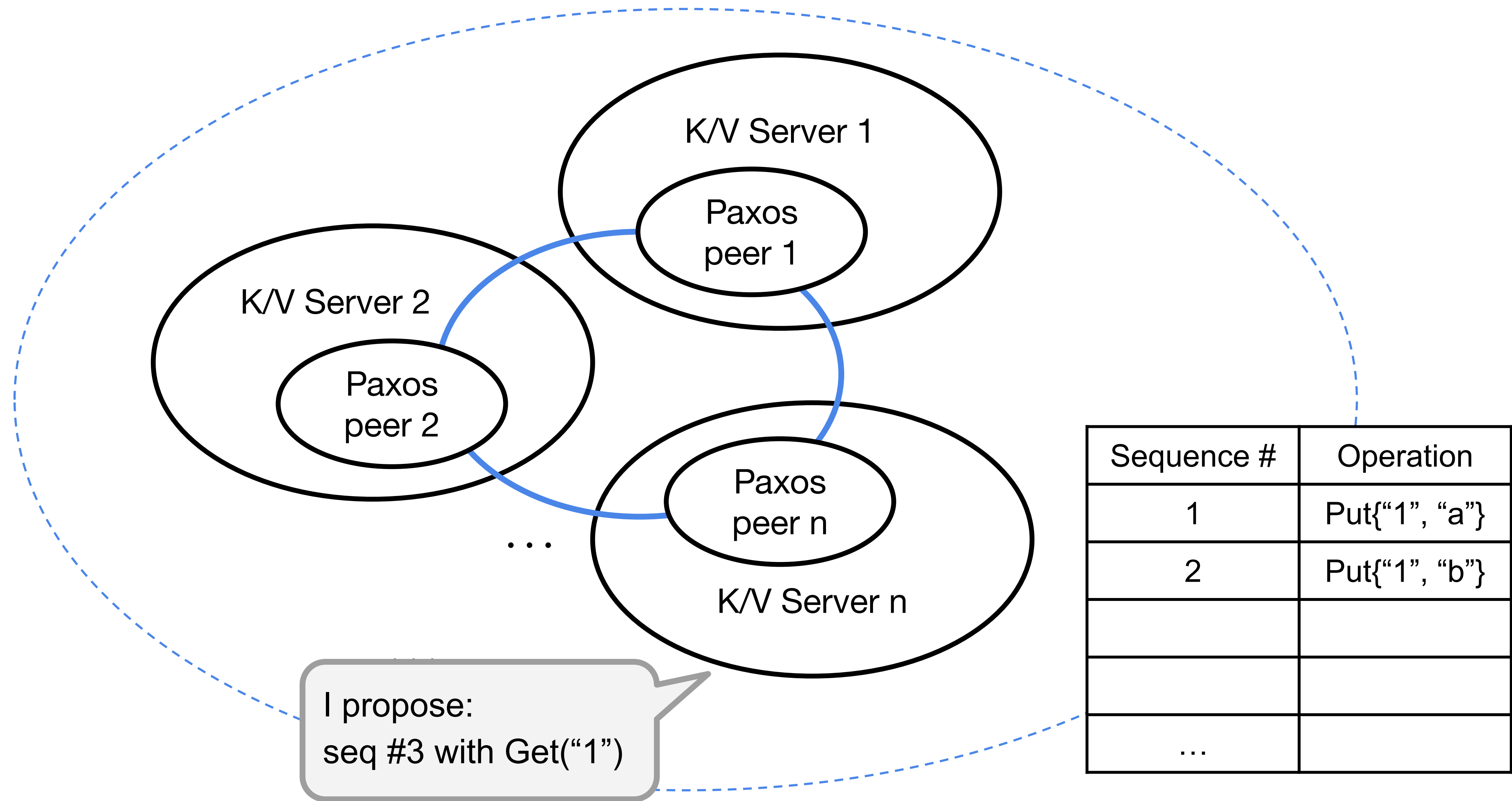


# Paxos-based Key/Value Service



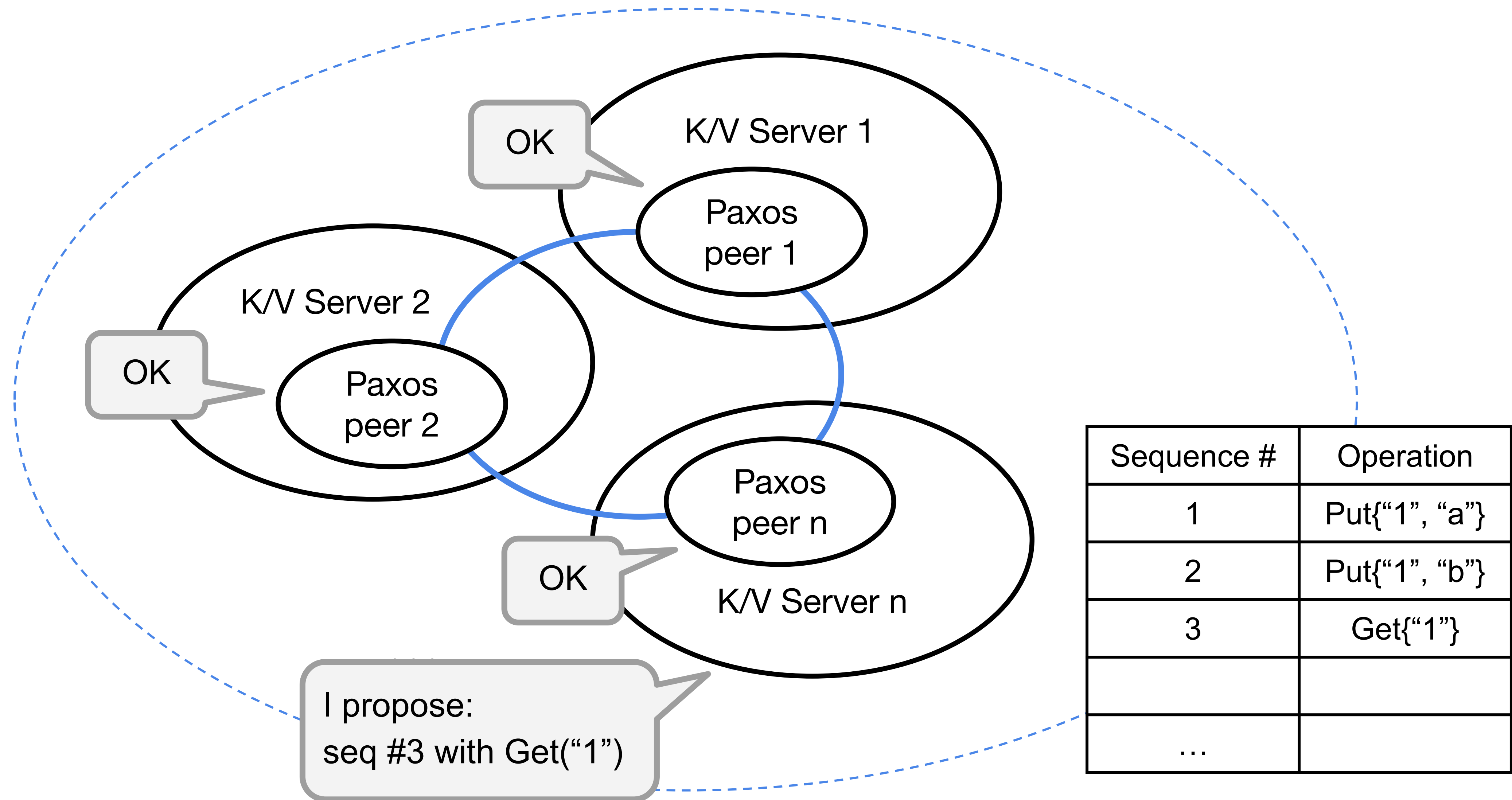


# Paxos-based Key/Value Service





# Paxos-based Key/Value Service



# Overview of the system

- Consist of the following players: clients, key/value servers (i.e., kvpaxos servers), and Paxos peers.
  - Clients send Put(), PutHash(), and Get() RPCs to kvpaxos servers.
  - Each kvpaxos server contains a key/value database and a Paxos peer.
- Part A: Paxos
  - Implement a Paxos library (src/paxos/paxos.go).
- Part B: Paxos-based Key/Value Server
  - Build kvpaxos, a fault-tolerant key/value storage system using Paxos.
  - You'll modify client.go, common.go, server.go inside src/kvpaxos.

# Overview of the system

- Upon receiving a request (via RPCs) from some client, kvpaxos proposes an instance through its Paxos peer (via method calls).

```
type KVPaxos struct {  
    mu sync.Mutex  
    l net.Listener  
    me int  
    dead bool // for testing  
    unreliable bool // for testing  
    px *paxos.Paxos  
  
    // Your definitions here.  
}
```



# Overview of the system

- Upon receiving a request (via RPCs) from some client, kvpaxos proposes an instance through its Paxos peer (via method calls).

// Your implementation must support the following interfaces

px.Start(seq int, v interface{}) // start agreement on new instance

px.Status(seq int) (decided bool, v interface{}) // get info about an instance

px.Done(seq int) // ok to forget all instances <= seq

px.Max() int // highest instance seq known, or -1

px.Min() int // instances before this have been forgotten

- The Paxos peer then talk to each other (via RPCs) to achieve agreement on each operation.

# How Paxos solves consensus

- Instances:
  - Every instance contains a sequence numbers.
  - Each instance is either "decided" or “not yet decided”.
  - A decided instance has a value. If an instance is decided, then all the Paxos peers that are aware that it is decided will agree on the same value for that instance.
- In our case, instances are the entries in the log.

| Sequence # | Operation     |
|------------|---------------|
| 1          | Put{"1", "a"} |
| 2          | Put{"1", "b"} |
| 3          | Get{"1"}      |
|            |               |
| ...        |               |

# Advantages of Paxos

- Paxos peers will get the agreement right even if some replicas are unavailable (unreliable network, or isolated in network partitions).
  - As long as Paxos can assemble *a majority of replicas*, it can continue to process client operations.
  - Replicas that were not in the majority can catch up later by asking Paxos for operations that they missed.